

Artificial Neural Networks and the Singular Values Decomposition : from Theory to Machine Learning

Benjamin Carrel, Martin J. Gander

May 2020

Abstract

This is a reading work about machine learning supervised by Prof. M.Gander. The main idea is to introduce and understand the artificial neural networks. To do so, we go deeply through neural networks and the so called stochastic gradient descent algorithm. The article [2] shows that, under certain conditions, the optimal layers of a neural network are derived from the SVD of the data. Therefore, we wanted to verify the links between the linear algebra and the neural networks. For a classification problem, we compared the neural networks with another classifier based on PCA. Finally, we created a little game inspired by the popular game "Clash of Clans". We trained the neural network with many games using stochastic gradient descent and MATLAB's blackbox optimization `fminsearch` and we compared the results. This allowed us a better understanding of neural networks and having a global view from the mathematical theory to the application in machine learning.

Contents

1	Introduction	3
2	Artificial Neural Networks	3
2.1	A first example	3
2.2	Generalization	5
2.3	MATLAB code	7
3	Training a Neural Network	8
3.1	A first approach	8
3.2	Stochastic Gradient	9
3.2.1	Steepest descent	9
3.2.2	Stochastic gradient descent	11
3.2.3	Back-Propagation	14
3.2.4	MATLAB code	16
4	A link between Neural Networks and SVD	17
4.1	Context	17
4.2	Theorem and proof	18
4.2.1	Optimal parameters of the last layer	18
4.2.2	No nonlinear function at the hidden layer	20
4.2.3	The hidden layer contain a nonlinear function	21
4.2.4	Conclusion : all optimal parameters	22
4.3	Numerical experiments	22
5	Machine Learning	25
5.1	PCA vs fminsearch vs stochastic gradient	25
5.1.1	Comparison 1 : fminsearch vs stochastic gradient	25
5.1.2	Comparison 2 : SVD, fminsearch and stochastic gradient	26
5.2	Clash of Clans	30
5.2.1	The game	30
5.2.2	Probabilistic approach	31
5.2.3	AI approach	35
6	Conclusion	39

1 Introduction

Section 2 is an introduction to artificial neural networks. This section is inspired by [5] and [8]. After a small example we give the definition of an artificial neural network and a small MATLAB code.

Section 3 is the description of the stochastic gradient (descent) algorithm. This section is, again, inspired by [5] and [8]. First, we recall the steepest descent algorithm and then we give the stochastic gradient (descent) algorithm. We also give the construction using back-propagation and the MATLAB code.

Section 4 is a link between the singular value decomposition and the optimal weights of a neural network. This section is inspired by [2]. We begin with theoretical results and we finish with a numerical verification.

Section 5 is about machine learning. This part is not inspired by any paper. The ideas come from M.Gander. The first part is a comparison, in the context of classification, between artificial neural networks and a classifier based on the principal component analysis (PCA). The second part is an application of artificial neural networks with a little game inspired by Clash of Clans.

We finish with the conclusion in Section 6.

2 Artificial Neural Networks

2.1 A first example

Let's begin with a first simple neural network. The classical representation of a neural network has 3 main parts:

1. The input layer.
2. The hidden layer(s).
3. The output layer.

Figure 1 gives a simple representation of a neural network which takes 3 numbers as input and gives 1 number as output.

Each circle represents an artificial neuron. Here, each neuron of a layer is connected to the next layer of neurons. Mathematically, a neuron is just a number which represents "how much this neuron is important". However, what does a connection mean ? How to interpret multiple connections ? In fact, Figure 1 is a simplification of what is really happening. Let's dive into this example. The connections are computed in 3 steps:

1. "link = weight". Hence, we compute the weighted sum of all previous neurons.
2. Apply a bias to each neuron obtained.
3. Apply a function called activation function to each neuron obtained.

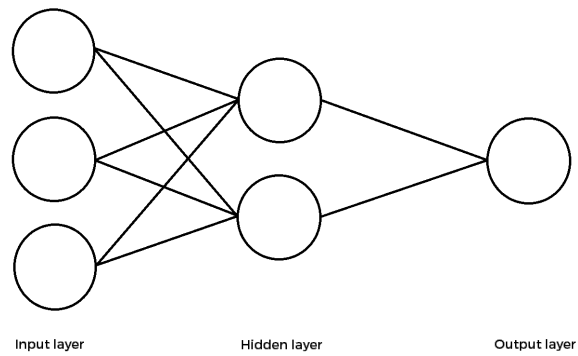


Figure 1: A simple neural network. It represents 3 inputs, 1 hidden layer made of 2 neurons, and 1 output.

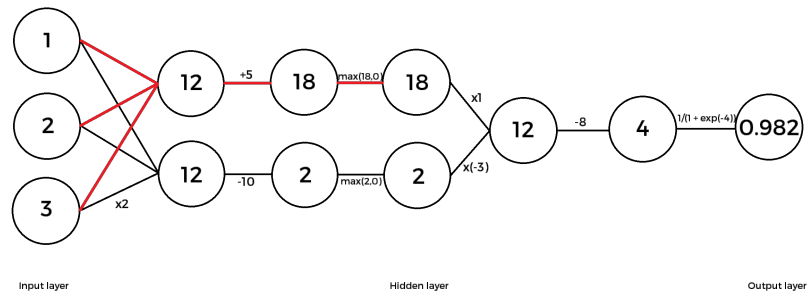


Figure 2: A more complete representation of the previous neural network. Numbers are given for the example of the computation in the text.

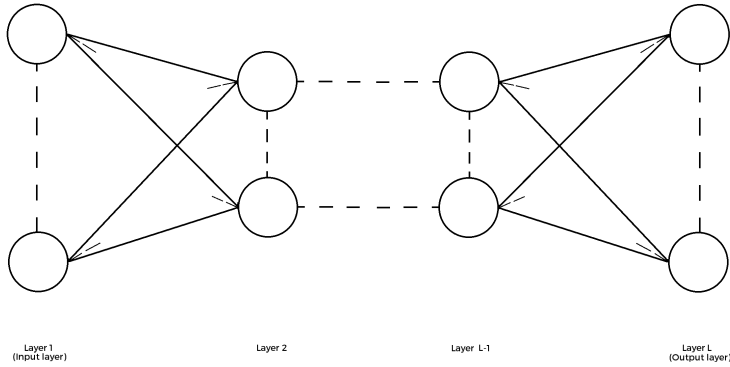


Figure 3: This figure represents a general neural network.

After these 3 steps, you get the next neuron layer. Figure 2 is the same neural network expanded explicitly with the 3 steps.

For an explicit example, Figure 2 is filled with numbers. Let's compute explicitly each step of the red path.

1. Here, all weights are 2. The weighted sum is: $2 * 1 + 2 * 2 + 2 * 3 = 12$.
2. Bias the result: $12 + 5 = 18$.
3. Here, activation function is $\max(x, 0)$ so we compute: $\max(18, 0) = 18$.

We get the value of the first neuron in the hidden layer. Do the same for the second neuron. Of course, weights and biases are independent. Then, do the same to obtain the output layer. Note that the activation function can be different at each layer. Here, the first activation function is $\max(x, 0)$ and the second is $\frac{1}{1+e^x}$. Finally, the output is 0.98. It may be interpreted as a probability or whatever you want to describe.

2.2 Generalization

Let's generalize this idea. A neural network can contain as many neurons and layers as we want. Figure 3 is a representation of a general neural network.

At this point, the magic of mathematics happens because the generalized model is very simple and everything is working very well.

- A layer ? It is a simple vector.
- The weighted sum ? It is a simple matrix multiplication.
- The biases ? Just add a vector.

- The activation function ? Just apply the function element-wise.

Let's do this formally. To simplify the reading, the notations are the same as in [5]. Suppose the network has L layers. The 1st layer is the input layer and the L th layer is the output layer. The layer 1 has n_1 neurons, the layer 2 has n_2 neurons and so on... , so that layer l has n_l neurons. Let $a_i^{[l]}$ be the i th neuron of the l th layer. Let $b_i^{[l]}$ be the i th bias of the l th layer. Let $W^{[l]}$ be the weighted matrix applied to the $(l-1)$ st layer. Let $\sigma_{[l]}$ be the activation function applied to the l th layer. Then, the neural network is entirely defined by

$$a^{[1]} = \text{input} \in \mathbb{R}^{n_1}, \quad (1)$$

$$a^{[l]} = \sigma_{[l]} \left(W^{[l]} a^{[l-1]} + b^{[l]} \right) \in \mathbb{R}^{n_l} \text{ for } l = 2, 3, \dots, L, \quad (2)$$

where

$$b^{[l]} \in \mathbb{R}^{n_l}, \quad (3)$$

$$W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}. \quad (4)$$

Finally, with $F : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_L}$ the application of the whole neural network, we get

$$\text{output} = F(\text{input}), \quad (5)$$

where

$$F(x) = \sigma_{[L]} \left(W^{[L]} \sigma_{[L-1]} \left(\dots \sigma_{[2]} \left(W^{[2]} \sigma_{[1]} \left(W^{[1]} x + b^{[1]} \right) + b^{[2]} \right) \dots \right) + b^{[L]} \right). \quad (6)$$

We refer to Wikipedia for typical activation functions used in machine learning. In the rest of the article, we will often use the sigmoid, and its derivative, defined by

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad (7)$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)). \quad (8)$$

We now go back to the previous simple example in Figure 1. We had a total of 3 layers, which implies 2 weighted matrices and 2 biases. In this particular case, we get

$$\begin{aligned} W^{[2]} &= \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix}, & W^{[3]} &= \begin{bmatrix} w_{11} & w_{12} \end{bmatrix}, \\ b^{[2]} &= \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}, & b^{[3]} &= \begin{bmatrix} b_1 \end{bmatrix}. \end{aligned}$$

An interesting exercise is to write each weight on each link on the representation. After that, it is easy to imagine a sparse neural network for fast computations with just few links. We can also imagine more complex oriented networks which could be used as neural network. It is exciting to think about all possibilities offered by these structures. Some people already observed that some structures are more efficient than others for specific problems. Finding such good structures is a part of the problem in the domain of artificial intelligence.

2.3 MATLAB code

Now, we can easily set-up a neural network from scratch in MATLAB. First, we define the shape of the network in a row vector. For example, the vector $[3, 2, 1]$ represents example in Figure 1. Then, we can create random weights and biases using the MATLAB function in Listing 1.

Listing 1: MATLAB function creating random weights and biases.

```

1 function [W,b]=RandomWeights(LayerSize)
2 % RANDOMWEIGHTS random neural network weights
3 % [W,b]=RandomWeights(LayerSize)
4 % LayerSize : vector of the size of each layer, including input ...
   and output
5 % i.e. [inputSize HiddenLayer1Size ... outputSize] (1 x nLayer)
6 % W : list of {nLayer} random matrices with appropriate size
7 % b : list of {nLayer} random biases with appropriate size
8 nLayer = length(LayerSize);
9 for k = 2:nLayer
10     W{k} = rand(LayerSize(k), LayerSize(k-1));
11     b{k} = rand(LayerSize(k),1);
12 end

```

Now, we have to define by hand the activation function we will use, and put it in a list called `activation`. Of course, if they are all the same (the sigmoid, for example), we can simply replace `activation{k}` by `sigmoid`. Then we can evaluate the prediction of the neural network for a given input using the MATLAB function in Listing 2.

Listing 2: MATLAB function applying the neural network.

```

1 function output = NeuralNetwork(input,activation,W,b)
2 % NEURALNETWORK neural network function
3 % output = NeuralNetwork(input,activation,W,b)
4 % input : a vector or the data, size n.1 x nData
5 % activation : list of nLayer functions to apply to hidden layers
6 % W : list of weighted matrices
7 % b : list of biases
8 % output : The output of the neural network
9 nLayer = length(W);
10 [n,nData] = size(input);
11 u = ones(nData,1)';

```

```

12 L{1} = input;
13 for k = 2:nLayer
14     L{k} = activation{k}(W{k}*L{k-1} + b{k}*u); % compute each layer
15 end
16 output = L{end};

```

3 Training a Neural Network

3.1 A first approach

In nature, all biological neural networks are trained by the same rule : trial & error. The idea is always to stimulate the neural network, and associate this stimulation to a positive or a negative feeling : win or lose, 1 or 0. Of course, a very complex neural network like the human brain can create much more sophisticated results. But this is the idea. Try and fail (or win) and accumulate experience over time.

Artificial neural networks are inspired by this model and the same is true for training them. This is the classical idea present everywhere in applied mathematics : minimize the error. Suppose we have a set of data, ie, a set S of couples $(input, output)$. We want to find a function F such that

$$F(input) \approx output \text{ for all } (input, output) \in S. \quad (9)$$

The error is defined by $e = F(input) - output$. We want to minimize the L^2 -norm $\|\cdot\|$. We are interested in the function F itself, so we want to find

$$\operatorname{argmin}_{F \in \mathcal{F}} \sum_{(x,y) \in S} \|F(x) - y\|^2, \quad (10)$$

where $\mathcal{F}$ denotes the set of all existing functions. Of course, there are an infinite number of functions that perform the best error, ie, 0. For example, we can define by hand $F(x) = y$ for each $(x, y) \in S$. Obviously, we don't want such functions. We want to find functions that, in a certain sense, are good to generalize to unseen data. We also often ask for the functions to be quite smooth, observing that the nature seems to be smooth in most processes.

In the context of neural networks, we impose the function F to be a certain neural network, with a predefined number of layers and neurons. The idea remains the same : minimize the cost. Given a dataset S , the problem is to

$$\operatorname{minimize}_{W^{[i]}, b^{[i]}} \left(\frac{1}{2|S|} \sum_{(x,y) \in S} \|F(x) - y\|^2 \right), \quad (11)$$

where F is the function (6) defined before.

Remark 1. F is a neural network : it is a big assumption. Or not ? Remember, we can define as many layers and neurons as we want, and define

the activation functions as we want so it seems reasonable that we can cover a large proportion of all possible functions. Because we predefined the shape of the neural network, we can compute how many parameters (degrees of freedom) we have. The parameters are the weights and the biases,

$$\begin{aligned} W^{[2]} &\in \mathbb{R}^{(n_2 \times n_1)}, & \dots, & & W^{[L]} &\in \mathbb{R}^{(n_L \times n_{L-1})}, \\ b^{[2]} &\in \mathbb{R}^{n_2}, & \dots, & & b^{[L]} &\in \mathbb{R}^{n_L}. \end{aligned}$$

Hence, the total number of parameters is

$$\#parameters = \sum_{i=2}^L (n_i * n_{i-1} + n_i). \quad (12)$$

To give an idea how large it is, consider a naive neural network that takes a picture of 400×400 pixels as input, ie, a vector of size $n_1 = 160000$. Considering the neural network with shape $[n_1, n_2, n_3, n_4] = [160000, 30, 30, 10]$ whose aim is to classify the images in 10 different classes. We get 4801270 parameters. This is enormous. So many parameters are difficult, if not impossible, to optimize. We can force sparsity to reduce the number of parameters: that's what the convolutional neural networks (CNN) do. (See [5] for more details.) In practice, we don't want to find the global minimum: we may overfit the data. We want a minimum that fits well to unseen data, so we need to use a test set, and some techniques like cross-validation. We can also imagine to regularize the network with a penalty term that penalizes the number of parameters and/or the norm of the weighted matrices.

We will focus on the main question: How to optimize a given neural network? Of course, we can do it using a classical black-box optimization code: it would find one of the minima. However, the computation may take too much time, and sometimes not converge as we want to. Worse, the minimum found performs bad on unseen data. Thankfully, a specific algorithm exists, the so-called stochastic gradient descent (or stochastic gradient) algorithm.

3.2 Stochastic Gradient

3.2.1 Steepest descent

First, we recall the steepest descent algorithm (without any parameter).

Algorithm 1 Steepest descent. Goal : minimize f .

Require: $\nabla f, x_1, \eta > 0, N$
for $k = 1$ to $N-1$ **do**
 $x_{k+1} \leftarrow x_k - \eta \nabla f(x_k)$
end for
return x_N

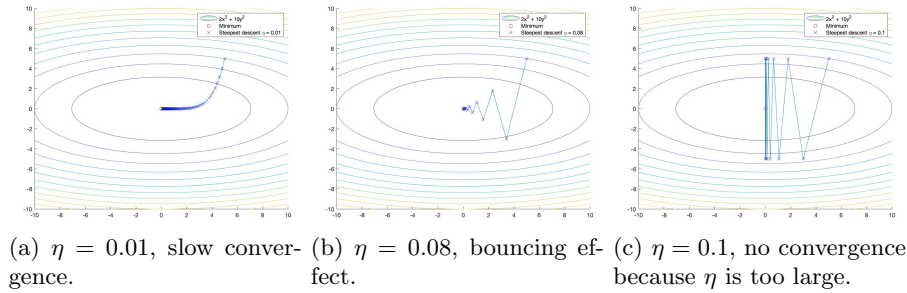


Figure 4: Steepest descent example : typical behaviors.

In the context of neural networks and machine learning, the term η is called the learning rate. It can be constant or not. In Listing 3 is a small MATLAB code applying the steepest descent algorithm to the function $f(x, y) = 2x^2 + 10y^2$.

Listing 3: Steepest descent example.

```

1  %% Define the function
2  xs = linspace(-10,10,100);
3  ys = linspace(-10,10,100);
4  [X,Y] = meshgrid(xs,ys);
5  f = 2*X.^2 + 10*Y.^2;
6
7  %% Steepest descent algorithm
8  % Input
9  x1 = [5,5]';
10 grad_f = @(x) [4*x(1), 20*x(2)]';
11 eta = 0.01;
12 N = 100;
13 % Steepest descent loop
14 x = zeros(2,N);
15 x(:,1) = x1;
16 for i = 1:N-1
17     x(:,i+1) = x(:,i) - eta * grad_f(x(:,i));
18 end
19
20 %% Plot the result
21 hold on
22 contour(xs,ys,f)
23 plot(0,0,'ro')
24 for i = 1:N
25     plot(x(1,i), x(2,i), 'bx')
26 end
27 line(x(1,:), x(2,:))
28 legend('2x^2 + 10y^2', 'Minimum', 'Steepest descent \eta = 0.01')

```

Figure 4 shows the results for 3 different learning rates η . As we can see, steepest descent is very inefficient. Of course, it is possible to optimize it in many cases. More powerful algorithms exist so why is it interesting? Stochastic gradient descent is based on it and has very nice properties when applied to a

neural network.

3.2.2 Stochastic gradient descent

In the context of neural networks, the cost function to minimize is given by (11), i.e. we want to

$$\underset{W^{[i]}, b^{[i]}}{\text{minimize}} \left(\frac{1}{2|S|} \sum_{(x,y) \in S} \|F(x) - y\|^2 \right).$$

Hence, the gradient is a sum, over all the data, of potential big matrix-vectors multiplications. In other words, the more data we have, the more expensive the gradient is. Maybe we have a lot of data. So, what can we do ? The answer is very simple : compute the gradient over only one data. This "partial" gradient is now much cheaper to compute. However, the information about the descent direction is incomplete. We can state the stochastic gradient algorithm.

Algorithm 2 Stochastic gradient descent. Goal : minimize $f = \sum_{s \in S} F_s$.

Require: $\nabla F, x_1, S, \eta > 0, N$
for $k = 1$ to $N-1$ **do**
 Take a random $s \in S$
 $x_{k+1} \leftarrow x_k - \eta \nabla F_s(x_k)$
end for
return x_N

Before applying the algorithm 2 to a neural network, we illustrate it with a simpler function. We need a function with many local minimum. Moreover, it must be a sum of function in order to apply stochastic gradient. A good function is given by summing gaussian functions. Hence, we consider the following function.

$$f(x, y) = \sum_{i=1, \dots, 4} F_i(x, y), \quad (13)$$

where

$$\begin{aligned} F_1(x, y) &= -e^{-(x-1)^2}, F_2(x, y) = -e^{-(x+1)^2}, \\ F_3(x, y) &= -e^{-(y-1)^2}, F_4(x, y) = -e^{-(y+1)^2}. \end{aligned}$$

This function has 4 minimum distributed in the 4 points: $[-1, -1]; [-1, 1]; [1, -1]; [1, 1]$. The "partial" gradients are given by

$$\nabla F_1 = \begin{pmatrix} 2(x-1)e^{-(x-1)^2} \\ 0 \end{pmatrix}, \nabla F_2 = \begin{pmatrix} 2(x+1)e^{-(x+1)^2} \\ 0 \end{pmatrix}, \quad (14)$$

$$\nabla F_3 = \begin{pmatrix} 0 \\ 2(y-1)e^{-(y-1)^2} \end{pmatrix}, \nabla F_4 = \begin{pmatrix} 0 \\ 2(y+1)e^{-(y+1)^2} \end{pmatrix}. \quad (15)$$

The Listing 4 is a MATLAB code applying stochastic gradient and steepest descent to this example. It produces Figure 5.

Listing 4: Stochastic gradient example.

```

1  %% Define the function
2  rng(1234) % for reproducibility
3  xs = linspace(-2,2,100);
4  ys = linspace(-2,2,100);
5  [X,Y] = meshgrid(xs,ys);
6  F1 = -exp(-(X-1).^2);
7  F2 = -exp(-(X+1).^2);
8  F3 = -exp(-(Y-1).^2);
9  F4 = -exp(-(Y+1).^2);
10 f = F1 + F2 + F3 + F4;
11
12 %% Stochastic gradient descent algorithm
13 % Input
14 x1 = [-2,-2];
15 gradF{1} = @(x) [2*(x(1)-1)*exp(-(x(1)-1)^2), 0]';
16 gradF{2} = @(x) [2*(x(1)+1)*exp(-(x(1)+1)^2), 0]';
17 gradF{3} = @(x) [0, 2*(x(2)-1)*exp(-(x(2)-1)^2)]';
18 gradF{4} = @(x) [0, 2*(x(2)+1)*exp(-(x(2)+1)^2)]';
19 gradf = @(x) ...
    [2*(x(1)-1)*exp(-(x(1)-1)^2)+2*(x(1)+1)*exp(-(x(1)+1)^2), ...
     2*(x(2)-1)*exp(-(x(2)-1)^2)+2*(x(2)+1)*exp(-(x(2)+1)^2)]';
20
21 eta = 1;
22 N = 200;
23 % Stochastic gradient loop
24 x = zeros(2,N);
25 x(:,1) = x1;
26 for i = 1:N-1
27     s = randi(4);
28     x(:,i+1) = x(:,i) - eta * gradF{s}(x(:,i));
29 end
30 % Steepest descent loop
31 y = zeros(2,N);
32 y(:,1) = x1;
33 for i = 1:N-1
34     y(:,i+1) = y(:,i) - eta * gradf(y(:,i));
35 end
36 %% Plot the result
37 levels = [1/2,1,2,4];
38 figure(1)
39 hold on
40 contour(xs,ys,f)
41 for i = 1:N
42     plot(x(1,i), x(2,i), 'rx')
43     plot(y(1,i), y(2,i), 'bx')
44 end
45 line(x(1,:), x(2,:), 'Color','red')
46 line(y(1,:), y(2,:), 'Color','blue')
47 legend('Function f = F_{1} + F_{2} + F_{3} + F_{4}', 'Stochastic ...
    gradient \eta=1', 'Steepest descent \eta=1')

```

As we can see on Figure 5, the behavior of stochastic gradient and steepest

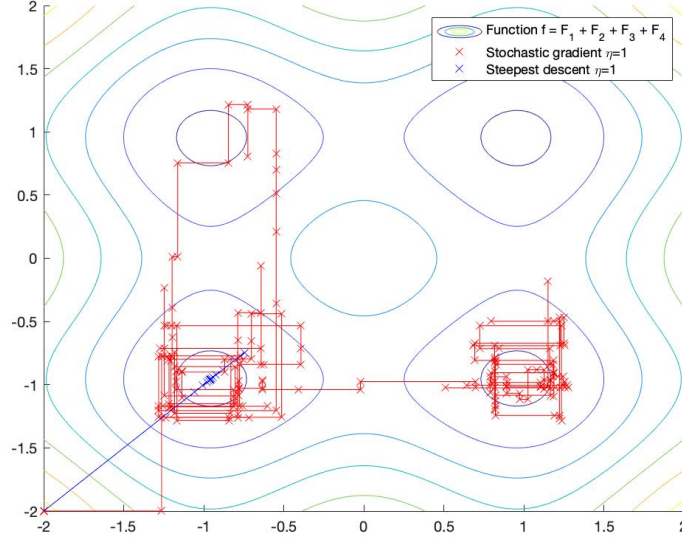


Figure 5: Run of stochastic gradient (red) and steepest descent (blue) with $\eta = 1$. The starting point is $[-2, -2]$ and we perform 200 iterations.

descent are completely different. Steepest descent is quickly stuck in a minimum while stochastic gradient is randomly pulled by each minimum. Hence, stochastic gradient explores the whole minimal area and that is exactly what we expect, find the best area to fit the data. Figure 5 also shows that we never know where stochastic gradient has stopped. That's why performing many runs of stochastic gradient is a good idea. In Section 5, the behavior of the cost function can be understood via Figure 5.

In the book [8], some methods with an adaptive learning rate η_k are described. In the same book, the following properties are promised.

"When we apply stochastic gradient to a neural network, we observe two miraculous properties :

1. Stochastic gradient converges quickly to a minimum, in less iterations than steepest descent.
2. The minimum found by stochastic gradient performs well on unseen data.

If the first one is imaginable, the second one is really a miracle."

For the moment, we don't understand why this should be true. But observations are unanimous : stochastic gradient is the best algorithm for neural networks. This observation is clear in the machine learning example in Section 5.

3.2.3 Back-Propagation

At this moment, we need an efficient algorithm to train a neural network. Remember the cost function (11). Considering only one data, the cost function simply becomes

$$C = \frac{1}{2} \|a^{[L]} - y\|^2 \text{ for a fixed } (x, y) \in S. \quad (16)$$

For this fixed couple $(x, y) \in S$, C is a function of the weights and biases which are included in the notation $a^{[L]}$. Hence, we need to compute and update the partial derivatives in direction of all weights and biases

$$\frac{\partial C}{\partial w_{jk}^{[l]}}, \frac{\partial C}{\partial b_j^{[l]}}, \text{ for } l = 2, 3, \dots, N.$$

In order to do that, it is convenient to introduce two notations.

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]} \in \mathbb{R}^{[n_l]} \text{ for } l = 2, 3, \dots, N, \quad (17)$$

$$\delta_j^{[l]} = \frac{\partial C}{\partial z_j^{[l]}} \text{ for } 1 \leq j \leq n_l \text{ and } 2 \leq l \leq L. \quad (18)$$

The expression (18) is often called the error in the j th neuron at layer l . The word error is ambiguous here, but it is important to keep in mind that $\delta_j^{[l]}$ measures the sensitivity of the cost function with respect to the weighted input for neuron j at layer l .

Remark 2. Using the definition and this notation, we get

$$a^{[l]} = \sigma_{[l]} \left(z^{[l]} \right) \text{ for } l = 2, 3, \dots, L. \quad (19)$$

The following lemma will be very useful to compute the gradient efficiently.

Lemma 1. Using the notation $\cdot *$ for the component-wise product of two vectors, we have

$$\delta^{[L]} = \sigma'_{[L]} \left(z^{[L]} \right) \cdot * \left(a^{[L]} - y \right), \quad (20)$$

$$\delta^{[l]} = \sigma'_{[l]} \left(z^{[l]} \right) \cdot * \left(W^{[l+1]} \right)^T \delta^{[l+1]} \quad \text{for } 2 \leq l \leq L-1, \quad (21)$$

$$\frac{\partial C}{\partial b_j^{[l]}} = \delta_j^{[l]} \quad \text{for } 2 \leq l \leq L, \quad (22)$$

$$\frac{\partial C}{\partial w_{jk}^{[l]}} = \delta_j^{[l]} a_k^{[l-1]} \quad \text{for } 2 \leq l \leq L. \quad (23)$$

Proof. The proof is simple but quite long and technical. So we refer to [5] for the complete proof. However, the following paragraph explains in words what is happening. $\square$

A couple of comments are necessary to understand the depth of this lemma. Remember that we first compute the neural network in a forward pass, ie, we compute $a^{[1]}, z^{[2]}, a^{[2]}, z^{[3]}, \dots, a^{[L]}$ in order. Thus, we directly have access to $\delta^{[L]}$ with (20). Then, using (21) we can compute $\delta^{[L-1]}, \delta^{[L-2]}, \dots, \delta^{[2]}$ in order. So we compute them in a backward pass. Having all $\delta^{[l]}$ and using (22) and (23), we also get all partial derivatives of C . Computing the gradients in this way is known as *back-propagation*.

To gain further understanding of the derivatives computed in (22) and (23), it is useful to recall the definition of partial derivatives. The quantity $\partial C / \partial w_{jk}^{[l]}$ measures how C changes for a small perturbation in $w_{jk}^{[l]}$. It is important to notice that a perturbation in a weight of a certain layer $l \leq L$ will not have any consequences on the previous layers $l-1, l-2, \dots, 2$, which explains this "backward" computing. Then, recall that $\delta_j^{[l]}$ measures the sensitivity of C with respect to the inputs of the j th neuron at layer l . This neuron is fed by the weights $w_{jk}^{[l]}$ multiplied by the element of the previous layer $a_k^{[l-1]}$. To summary, $\delta_j^{[l]}$ measures the sensitivity of C which is obtained after computing $w_{jk}^{[l]} a_k^{[l-1]} + \text{constant}$, so it makes sense that

$$\frac{\partial C}{\partial w_{jk}^{[l]}} = \delta_j^{[l]} a_k^{[l-1]}.$$

Similarly for the biases, the neurons are fed by $b_j^{[l]} + \text{constant}$, which explain why

$$\frac{\partial C}{\partial b_j^{[l]}} = \delta_j^{[l]}.$$

The product with diagonal matrices can replace the component-wise product in Lemma 1. Let $D^{[l]} \in \mathbb{R}^{n_l \times n_l}$ be the diagonal matrix where the (i, i) entry is given by $\sigma' \left(z_i^{[l]} \right)$. Then,

$$\begin{aligned} \delta^{[L]} &= \sigma'_{[L]} \left(z^{[L]} \right) \cdot * \left(a^{[L]} - y \right) && \text{becomes } \delta^{[L]} = D^{[L]} (a^{[L]} - y), \\ \delta^{[l]} &= \sigma'_{[l]} \left(z^{[l]} \right) \cdot * \left(W^{[l+1]} \right)^T \delta^{[l+1]} && \text{becomes } \delta^{[l]} = D^{[l]} \left(W^{[l+1]} \right)^T \delta^{[l+1]}. \end{aligned}$$

As derived in Lemma 1, the partial derivatives of C are given by

$$\frac{\partial C}{\partial b_j^{[l]}} = \delta_j^{[l]}, \tag{24}$$

$$\frac{\partial C}{\partial w_{jk}^{[l]}} = \left(\delta^{[l]} \left(a^{[l-1]} \right)^T \right)_{jk}. \tag{25}$$

We finally have all materials for applying stochastic gradient descent to a neural network. First, we need to define the neural network, ie, define L layers, $W^{[l]}$ the weights and $b^{[l]}$ the biases. We also need to define the learning rate η and the total number of iterations N . At each iteration, we will:

1. Choose one random couple $(x, y) \in S$.
2. Compute the forward pass to get $z^{[l]}, a^{[l]}, D^{[l]}$ for $l = 2, \dots, N$.
3. Compute the backward pass to get $\delta^{[l]}$ for $l = L, L - 1, \dots, 2$.
4. Update the weights $W^{[l]}$ and biases $b^{[l]}$.

The algorithm 3 is the so-called stochastic gradient with back-propagation applied to a neural network.

Algorithm 3 Stochastic gradient for neural networks.

Require: Initial $W^{[l]}$ and $b^{[l]}, \eta > 0, N$

for k = 1 to N **do**

Choose a random couple $(x, y) \in S$.

(x, y) is the current training data couple.

$a^{[1]} = x$

for l=2 to L **do**

$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$

$a^{[l]} = \sigma_{[l]}(z^{[l]})$

$D^{[l]} = \text{diag}(\sigma'_{[l]}(z^{[l]}))$

end for

$\delta^{[L]} = D^{[L]}(a^{[L]} - y)$

for l = L-1 downto 2 **do**

$\delta^{[l]} = D^{[l]}(W^{[l+1]})^T \delta^{[l+1]}$

end for

for l = L downto 2 **do**

$W^{[l]} \rightarrow W^{[l]} - \eta \delta^{[l]}(a^{[l-1]})^T$

$b^{[l]} \rightarrow b^{[l]} - \eta \delta^{[l]}$

end for

end for

return x_N

Note that the dimensions of each $W^{[l]}$ and $b^{[l]}$ are not necessarily the same, we cannot use three-dimensional arrays. Depending on the language we are using, we can use cell arrays or structure arrays. In MATLAB, we used cell arrays.

3.2.4 MATLAB code

Listing 5 shows a MATLAB function which applies stochastic gradient to a given neural network. This function is written in a compact way. We can, for example, add a monitor (ie, compute the error at each step) and add a tolerance level. This is what we used to inspect the behavior of the algorithm. Moreover, the typical cost generated by stochastic gradient are shown in the section 5.

Listing 5: Stochastic gradient function

```

1 function [W, b] = StochasticGradient(N, eta, W, b, sigma, ...
    sigma_prime, data_in, data_out)
2 % STOCHASTICGRADIENT stochastic gradient descent algorithm
3 % [W, b] = StochasticGradient(N, eta, W, b, sigma, sigma_prime, ...
    data_in, data_out)
4 % N : number of iterations
5 % eta : learning rate
6 % W : list of weighted matrices
7 % b : list of bias vectors
8 % sigma : list of activation functions
9 % sigma_prime : list of derivatives of activation functions
10 % data_in : matrix of input data
11 % data_out : matrix of output data
12 % W : list of weighted matrices after training
13 % b : list of bias vectors after training
14 [n_data] = size(data_in); % quantity of data
15 L = length(W); % number of layers
16 for counter = 1:N % iteration loop
17     k = randi(n_data); % random sample
18     x = data_in(:,k); % corresponding data
19     a{1} = x;
20     for l = 2:L % forward pass
21         z{1} = W{1}*a{l-1}+b{1};
22         a{1} = sigma{1}(z{1});
23         D{1} = diag(sigma_prime{1}(z{1}));
24     end % forward end
25     Delta{L} = D{L}*(a{L}-data_out(:,k)); % backward pass
26     for l = L-1:-1:2
27         Delta{1} = D{1}*W{1+1}'*Delta{l+1};
28     end % backward end
29     for l = L:-1:2 % gradient step
30         W{1} = W{1} - eta*Delta{1}*a{l-1}'; % update weights
31         b{1} = b{1} - eta*Delta{1}; % update biases
32     end
33 end

```

4 A link between Neural Networks and SVD

4.1 Context

Until now, we focused on a specific class of neural networks. The nonlinear computing elements were arranged in a feed-forward layered structure. This class is called multilayer perceptrons (MLP). An even more specific case is when the input size is equal to the output size. It is called auto-association. The purpose of auto-association is to compress the data through smaller layers. The article [2] from H. Bourlard and Y. Kamp shows that the optimal weights can be derived from the singular value decomposition (SVD) in the context of auto-association. This result is interesting in many ways. First, we can find the global minimum which is rarely found by the stochastic gradient algorithm. Second, since the optimal values are derived from linear algebra, it makes the

nonlinear functions at hidden layers unnecessary. Finally, the analysis allows a better understanding of the role of each parameter.

4.2 Theorem and proof

Let's begin by some notations. Let n be the size of input vectors. We are talking about auto-association so n is also the size of the output vectors. Let N be the quantity of data. Let $X, Y \in \mathbb{R}^{n \times N}$ being respectively the matrices of input and output data. At first, we begin with only one hidden layer of size $p < n$, and stock the value of the layer for each input in the matrix $H \in \mathbb{R}^{p \times N}$. Hence, the shape of the neural network is $[n, p, n]$.

Explicitly, we have

$$\begin{aligned} X &= [x_1, x_2, \dots, x_N] & \text{where } x_i \in \mathbb{R}^n, \\ H &= [h_1, h_2, \dots, h_N] & \text{where } h_i \in \mathbb{R}^p, \\ Y &= [y_1, y_2, \dots, y_N] & \text{where } y_i \in \mathbb{R}^n. \end{aligned}$$

Let u be the N -vector of ones and F the activation function applied at the hidden layer. Finally, let C be the cost. Using the definition of a neural network, we can relate X, H and Y by

$$B = W_1 X + b_1 u^T, \quad (26)$$

$$H = F(B), \quad (27)$$

$$Y = W_2 H + b_2 u^T, \quad (28)$$

$$C = \|X - Y\|_F^2. \quad (29)$$

Before continuing, it is important to observe two things:

1. We suppose that we don't apply any nonlinear function to the output, ie, the last layer of the network.
2. We are working in the Frobenius norm because the derivations are easier.

The training problem is however the same, minimize C with respect to the parameters W_1, W_2, b_1, b_2 .

4.2.1 Optimal parameters of the last layer

Observing that, by (28), $C = \|X - Y\|_F^2 = \|X - W_2 H - w_2 u^T\|_F^2$ we can derive the following Lemma 2.

Lemma 2. *The optimal parameters $\hat{W}_2, \hat{H}'$ and $\hat{b}_2$ are given by*

$$\hat{W}_2 = U_p T^{-1}, \quad (30)$$

$$\hat{H}' = T \Sigma_p V_p^T, \quad (31)$$

$$\hat{b}_2 = \frac{1}{N} (X - \hat{W}_2 \hat{H}') u. \quad (32)$$

T is an arbitrary non singular $p \times p$ matrix, $U_p \Sigma_p V_p^T$ is the rank p approximation of $X' = X(I - uu^T/N) = U_n \Sigma_n V_n^T$, and $H' = H(I - uu^T)$.

Proof. The cost is given by

$$C = \|X - W_2 H - b_2 u^T\|_F^2, \quad (33)$$

so the definition of the Frobenius norm $\|A\|_F^2 = \text{tr}(AA^T)$ provides the minimum of C with respect to b_2 when

$$\hat{b}_2 = \frac{1}{N} (X - W_2 H) u. \quad (34)$$

Inserting (34) into (33), we get

$$\begin{aligned} C &= \left\| X - W_2 H - \frac{1}{N} (X - W_2 H) u u^T \right\|_F^2 \\ &= \|X(I - uu^T/N) - W_2 H(I - uu^T/N)\|_F^2 \\ &= \|X' - W_2 H'\|_F^2. \end{aligned} \quad (35)$$

In view of the fact that $W_2 \in \mathbb{R}^{n \times p}$ has a maximum rank $p < n$, the product $W_2 H'$ has also a maximum rank of $p < n$. Hence, the optimal $W_2 H'$ is given by the rank p approximation of X' (Golub 1968 [4] and Stewart 1973 [7]). In other words, if we denote the SVD

$$X' = U_n \Sigma_n V_n^T, \quad (36)$$

then optimal product is the rank p approximation, given by

$$\hat{W}_2 \hat{H}' = U_p \Sigma_p V_p^T, \quad (37)$$

where $\Sigma_p = \text{diag}[\sigma_1, \dots, \sigma_p]$ the p first singular values, and U_p (resp. V_p) is formed by the first p columns in U_n (resp. V_n). Consequently, we get the result given in the lemma

$$\hat{W}_2 = U_p T^{-1}, \quad \hat{H}' = T \Sigma_p V_p^T, \quad (38)$$

for an arbitrary non singular matrix $T \in \mathbb{R}^{p \times p}$. $\square$

Lemma 2 needs a couple of comments.

- Let μ_X denote the average of the input data, i.e., $\mu_X = \frac{1}{N} X u$. Similarly, $\mu_Y = \frac{1}{N} Y u$ is the average of the output data. Putting (34) into (28), the optimal bias $\hat{b}_2$ insures that

$$\mu_X = \mu_Y. \quad (39)$$

In other words, the average of input and output vectors are equal. Moreover, the effect of the bias $\hat{b}_2$ is to reduce the problem to zero-average pattern.

- In the special case where all input vectors are identical, ie, $X = \mu_X u^T$, then, $X' = 0$ so $\hat{W}_2 \hat{H}' = 0$. Thus, using Lemma 2

$$\hat{b}_2 = \frac{1}{N} (X - W_2 H) u = \frac{1}{N} X u = \mu_X,$$

the output is exactly the input.

- One can show that the covariance of the output vectors $y_1, y_2, \dots, y_N$ is the best rank p approximation of the covariance of the input vectors $x_1, x_2, \dots, x_N$ and, in this sense, the auto-associative MLP is nothing but an indirect way of performing data compression by the Karhunen-Loève transform on zero-average data (Ahmed and Rao 1975 [1]).
- The derivation of $\hat{H}'$ is completely independent from the function F .

Question : Is it possible to get this optimal $\hat{H}'$ by linear transformation ? What if we apply a nonlinear transformation ?

Answer : Yes and, under some assumptions, it is also possible with a non-linear transformation, which becomes useless. We will prove it in the following.

4.2.2 No nonlinear function at the hidden layer

Begin by supposing that we don't apply any nonlinear function at the hidden layer, i.e., $F(B) = B$. We have to show that $\hat{H}' = T \Sigma_p V_p^T$ can be generated in accordance to his definition. Hence, we get

$$\hat{H} = \hat{B} \stackrel{(26)}{=} \hat{W}_1 X + \hat{b}_1 u^T \quad (40)$$

$$\iff \hat{H} (I - uu^T/N) = (\hat{W}_1 X + \hat{b}_1 u^T) (I - uu^T/N)$$

$$\iff \hat{H}' = \hat{W}_1 X' + \hat{b}_1 u^T (I - uu^T/N) = \hat{W}_1 X' + 0 \quad (41)$$

$$\stackrel{Lem2}{\iff} T \Sigma_p V_p^T = \hat{W}_1 X'$$

$$\stackrel{\star}{\iff} T U_p^T X' = \hat{W}_1 X'$$

$$\implies \hat{W}_1 = T U_p^T. \quad (42)$$

The equivalence $\star$ comes from

$$U_p^T X' = U_p^T U_n \Sigma_n V_n^T = \Sigma_p V_p^T. \quad (43)$$

We derived all parameters of the neural network. In particular, observe that (41) implies that $\hat{b}_1$ is arbitrary.

To summarize, when there is no nonlinear function at hidden layer, the optimal parameters are given by

$$\hat{b}_1 = arbitrary \quad \in \mathbb{R}^p, \quad (44)$$

$$\hat{W}_1 = T U_p^T \quad \in \mathbb{R}^{p \times n}, \quad (45)$$

$$\hat{b}_2 = (I - U_p U_p^T) \mu_X - U_p T^{-1} b_1 \quad \in \mathbb{R}^n, \quad (46)$$

$$\hat{W}_2 = U_p T^{-1} \quad \in \mathbb{R}^{n \times p}, \quad (47)$$

where $\hat{b}_2$ is obtained by inserting (42) into (40) and then by inserting (47) and (40) into (34). As we can see, except for $\hat{b}_1$ which is arbitrary, the SVD of $X' = X(I - uu^T/N)$ is present in each optimal parameter, which shows the proximity between the auto-association by MLP and linear algebra.

4.2.3 The hidden layer contain a nonlinear function

Now, suppose that a nonlinear function F is applied at the output of the hidden units. We will need only one assumption, F can be approximated by the linear part of its expansion when x is small, ie,

$$F(x) \approx \alpha_0 + \alpha_1 x, \text{ for } x \text{ small and } \alpha_1 \neq 0. \quad (48)$$

This assumption is verified for typical activation function like

$$\text{The asymmetric sigmoid : } F(x) = \frac{1}{1 + e^{-x}} \approx \frac{1}{2} + \frac{1}{4}x \text{ when } x \text{ small,}$$

$$\text{The symmetrical sigmoid : } F(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \approx 0 + \frac{1}{2}x \text{ when } x \text{ small.}$$

We will show that with minor modifications, the optimal values previously obtained still produce the expression for $\hat{H}'$. We define

$$\hat{W}_1 = \alpha_1^{-1} T U_p^T, \quad (49)$$

which imply by (26)

$$\hat{B} = \hat{W}_1 X + b_1 u^T = \alpha_1^{-1} T U_p^T X + b_1 u^T. \quad (50)$$

If we want to use the expansion (48), B should be small. But we are lucky because $\hat{b}_1$ and T are arbitrary. In particular, we can define $\hat{b}_1$ such that $\mu_B = \frac{1}{N} B u = 0$ by setting

$$\hat{w}_1 = -\alpha_1^{-1} T U_p^T \mu_X. \quad (51)$$

Hence, we just need T to be non singular and small enough. With this chosen $\hat{w}_1$, we get

$$\hat{B} = \alpha_1^{-1} T U_p^T X' = \alpha_1^{-1} T \Sigma_p V_p^T. \quad (52)$$

Hence,

$$\hat{H} = F(\hat{B}) \approx \alpha_0 u u^T + T \Sigma_p V_p^T. \quad (53)$$

Then, using the definition,

$$\begin{aligned} \hat{H}' &= \hat{H}(I - uu^T/N) \\ &= (\alpha_0 uu^T + T \Sigma_p V_p^T)(I - uu^T/N) \\ \implies \hat{H}' &= T \Sigma_p V_p^T, \end{aligned}$$

which is exactly optimal $\hat{H}'$ derived above in lemma 2.

Finally, using $\hat{b}_2 = \frac{1}{N} (X - \hat{W}_2 \hat{H}) u$ and $\hat{W}_2 = U_p T^{-1}$, one can find

$$\hat{b}_2 = \mu_X - \alpha_0 U_p T^{-1} u. \quad (54)$$

Before the conclusion, a couple of comment is necessary.

- The same approach can be used for multiple hidden layers. The key operation is the SVD of $X' = X(I - uu^T/N)$ and its rank p approximation, where p is given by the last hidden layer. Of course, for each layer add, the degree of freedom in the choices of W_i and b_i is wider.
- Unfortunately, if a nonlinear function is used at the output layer, this approach breaks down. However, some interesting results can be derived and are shown to be closely connected with low rank realizations of prescribed sign matrices. (Delsarte and Kamp 1988 [3]).

4.2.4 Conclusion : all optimal parameters

To conclude, all derivations are summarized in Theorem 1.

Theorem 1. *In case of auto-association in MLP, suppose the smallest layer has size $p < n$ and the functions applied at hidden layer verify $F(x) \approx \alpha_0 + \alpha_1 x$ when x is small and $\alpha_1 \neq 0$. Then, if we don't apply a nonlinear function at the output layer, the optimal parameters of the MLP are given by*

$$\hat{W}_1 = \alpha_1^{-1} T U_p^T, \quad (55)$$

$$\hat{b}_1 = -\alpha_1^{-1} T U_p^T \mu_X, \quad (56)$$

$$\hat{W}_2 = U_p T^{-1}, \quad (57)$$

$$\hat{b}_2 = \mu_X - \alpha_0 U_p T^{-1} u. \quad (58)$$

where U_p, Σ_p, V_p^T correspond to the rank p approximation in the SVD of

$$X' = X(I - uu^T/N) = U_n \Sigma_n V_n^T,$$

and $T \in \mathbb{R}^{p \times p}$ a non singular scaling matrix, which has to be small enough to make the linear approximation of F , seen in (48).

4.3 Numerical experiments

This part was not done in the original article [2]. The goal of this section is to illustrate numerically Theorem 1. In order to do that, we simulated data.

Let $X \in \mathbb{R}^{20 \times 100}$ be the matrix of input data containing $N = 100$ vectors of size $n = 20$. It is auto-association so the output data is $Y \in \mathbb{R}^{20 \times 100}$. The network has one hidden layer of size $p = 5$, so the shape of the neural network is

[20, 5, 20]. The activation function applied to the hidden layer is the symmetrical sigmoid,

$$F(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \approx 0 + \frac{1}{2}x \text{ when } x \text{ small,}$$

$$F'(x) = \frac{2e^{-x}}{(1 + e^{-x})^2}.$$

Hence, in this example, we have $\alpha_0 = 0$ and $\alpha_1 = 1/2$, so Theorem 1 holds.

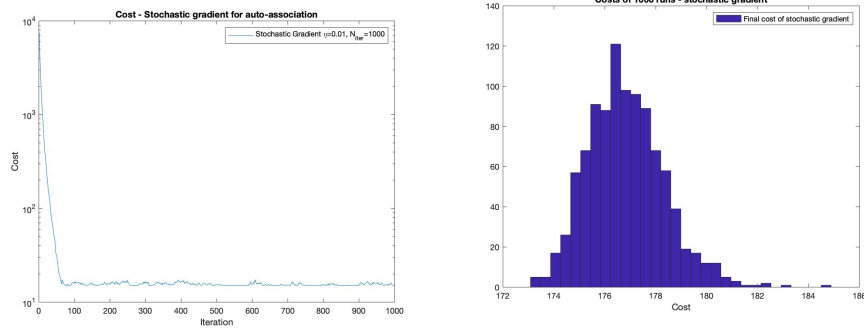
In Listing 6, after generating random data and the neural network, we compute the optimal weights as described in Theorem 1.

Listing 6: Numerical experiments of theorem 1

```

1  %% Definition of the problem and the neural network
2  rng(1234)                                % seed for reproducibility
3  n = 20; N = 100; p=5;                    % sizes of the problem
4  X = rand(n,N);                           % random input
5  Y = rand(n,N);                           % random output
6  layers = [n,p,n];                        % define the layers
7  u = ones(N,1);                           % vector of ones
8  mu_X = 1/N*X*u;                          % compute the mean
9  X_prime = X * (eye(N) - u*u'/N);         % define X'
10 [U,S,Vt] = svd(X_prime);                  % compute SVD of X'
11 Up = U(:,1:p);                           % p truncation of U
12 F = @(x) (1-exp(-x))./(1+exp(-x));        % activation function...
13 F_prime = @(x) 2*exp(-x)./(1+exp(-x)).^2; % ... and its derivative
14 id = @(x) x;                              % identity at output layer
15 act = {0,F,id};                           % list of functions
16 act_prime = {0,F_prime,id};               % and their derivatives
17
18 %% Optimal weights by the theorem 1
19 T = 1e-6*rand(p);                         % compute T...
20 T_inv = inv(T);                           % its inverse
21 W1_opt = 2 * T * Up';                     % optimal parameters
22 b1_opt = - 2 * T * Up' * mu_X;
23 W2_opt = Up * T_inv;
24 b2_opt = mu_X;
25 W_opt = {0, W1_opt, W2_opt};
26 b_opt = {0, b1_opt, b2_opt};
27
28 Y1 = NeuralNetwork(X,act,W_opt,b_opt);     % apply the neural network
29 C_opt = norm(X-Y1,'fro')^2                 % optimal cost
30
31 %% Stochastic gradient from optimal point
32 [W.sg, b.sg] = StochasticGradient(5,...
33     0.01,W_opt,b_opt,act,act_prime,X,Y); % apply stoch. grad.
34 Y2 = NeuralNetwork(X,act,W.sg,b.sg);
35 C.sg_opt = norm(X-Y2,'fro')^2              % final stoch. grad. cost
36
37 %% Stochastic gradient from random point
38 [W, b] = RandomWeights(layers);            % create random weights
39 [W.sg, b.sg] = StochasticGradient(1000,...
40     0.01,W,b,act,act_prime,X,Y);          % apply stoch. grad.
41 Y3 = NeuralNetwork(X,act,W.sg,b.sg);

```



(a) Evolution of the cost of one run of stochastic gradient. (b) Histogram of final costs of 1000 runs of stochastic gradient. Mean : $\mu = 176.8354$, Standard Deviation : $\sigma = 1.5105$.

Figure 6: Study of Stochastic Gradient for auto-association by MLP problem, with $n = 20$ and $p = 5$. The shape of the neural network is $[20, 5, 20]$.

```
42 C_sg_rand = norm(X-Y3, 'fro')^2 % final stoch. grad. cost
```

This code produced these three values

$$\begin{aligned} C_{opt} &= 102.9342, \\ C_{sg-opt} &= \text{NaN}, \\ C_{sg-rand} &= 175.1945. \end{aligned}$$

Of course, stochastic gradient is a random algorithm, so we need to perform Monte-Carlo simulations, see Figure 6. As we can see, the final cost produced by stochastic gradient C_{sg} is always much higher compared to C_{opt} , even playing with different learning rate η . Stochastic gradient is always stuck in a local minimum. We also tried to start stochastic gradient from the optimal weights and a neighborhood but it seems that the gradient is very complicated around the minimum. Indeed, after few iterations get NaN because the gradient (delta in stochastic gradient's code) is about 10^6 so the weights are also 10^6 . Then we need to compute $\exp(10^6)$ in the sigmoid, which gives NaN in MATLAB and the algorithm becomes useless. It means that the global minimum is literally impossible to obtain with the stochastic gradient algorithm.

To conclude this section, the results derived show that, under some assumptions, optimal weights for auto-association by MLP can be derived from linear algebra, making the use of nonlinear functions and many layers totally useless. However, the smallness required for the scaling matrix T is not well studied in the article [2], and the inversion of such a matrix may be problematic. Moreover, the following section shows that, without the assumptions of Theorem 1, the neural networks are much more complicated to analyse, but lead to much better results than linear algebra can do, see Section 5.

5 Machine Learning

5.1 PCA vs fminsearch vs stochastic gradient

Section 4 showed that, in case of auto-association, the optimal weights have a strong link with the SVD, reducing the neural network to a problem solved by linear algebra. Today, neural networks are often used in classification problems. We then wanted to compare neural networks for classification to a classifier obtained from linear algebra.

We also wanted to compare the predictions made by stochastic gradient and the predictions made by the fminsearch optimization blackbox routine in MATLAB. The goals are obviously to compare the performance of both algorithms in terms of

- time of computation,
- minimality of the cost found,
- the potential of the minimum reached to make predictions on unseen data.

We will consider three vectors, classified in two classes.

$$\begin{aligned}v_1 &= [1, 0, 0]^T \longleftrightarrow \text{class 0}, \\v_2 &= [0, 1, 0]^T \longleftrightarrow \text{class 0}, \\v_3 &= [0, 0, 1]^T \longleftrightarrow \text{class 1}.\end{aligned}\tag{59}$$

First, we need to simulate data. For each of these vectors, we create $n_{data} = 20$ similar vectors with random normally distributed noise. We suppose the output to be correct, i.e., each vector is correctly classified in the dataset. We also simulate a test set, created in the same way as the dataset and we chose $n_{test} = 10000$. It is a classification problem so the output needs to be between 0 and 1, so the activation functions are the sigmoid,

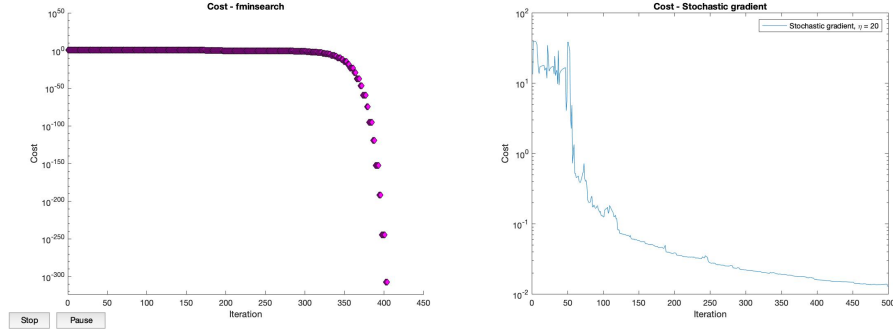
$$F(x) = \frac{1}{1 + e^{-x}},\tag{60}$$

$$F'(x) = F(x)(1 - F(x)).\tag{61}$$

5.1.1 Comparison 1 : fminsearch vs stochastic gradient

At first, we begin by a comparison of a single run of fminsearch and stochastic gradient. We consider a noise of scale 0.1 and a neural network of shape $[3, 2, 1]$. Figure 7 shows the cost produced by both algorithms. The following table is a summary of the results obtained.

Shape $[3, 2, 1]$	fminsearch	stochastic gradient $\eta = 20$
cputime	0.11	0.03
iterations	441	500
final error	0	0.0129
test set error	9.8489	2.6103



(a) cost produced by the blackbox fmin- (b) cost produced by stochastic gradient, search. $N_{iter} = 500, \eta = 20$.

Figure 7: fminsearch and stochastic gradient applied to a neural network of shape $[3, 2, 1]$ which aim to classify three vectors into 2 classes, see (59).

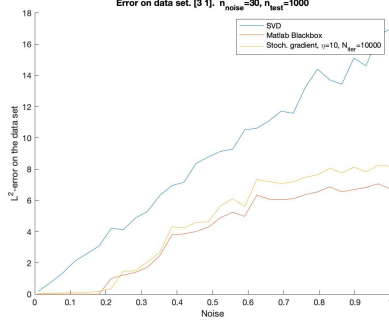
As expected, stochastic gradient is much faster compared to fminsearch. As promised, even though stochastic gradient does not reach the global minimum (here 0), it performs very well on unseen data. In next experiments we will perform Monte-Carlo simulations, i.e. many runs with the same initial weights, because stochastic gradient is a random algorithm, see Section 5.2.

5.1.2 Comparison 2 : SVD, fminsearch and stochastic gradient

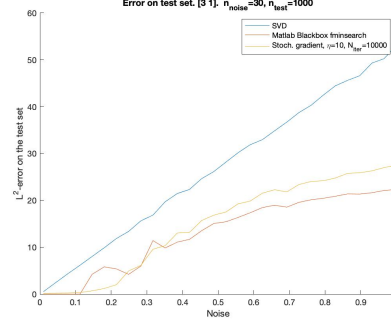
In this section, we used SVD as a linear classifier. To be more precise, we indirectly perform a PCA and then we use a classifier.

1. Create a matrix of data for each class.
2. Compute the SVD of this matrix.
3. The singular vectors with the highest singular values contain the informations of the class.
4. Project a new data on the corresponding subspace.
5. If the norm of the projection is high, the new data is classified in this class. Otherwise, it is not.

We want to classify the same three vectors in (59). We simulate $n_{data} = 100$ data for each vector and simulate a test set with $n_{test} = 1000$ data for each vector. We solve the problem for 30 different noises, from 0.01 to 1. In MATLAB, `noises = linspace(0.01, 1, 30)`. As we said before, stochastic gradient is a random algorithm so we performed 20 Monte-Carlo simulations and we keep the mean of the error. We need to use small layers and small datasets because doing all this is very computer intensive. (30 noises and 20 Monte-Carlo simulation means we perform 600 stochastic gradients with $N_{iter} = 1000$!)

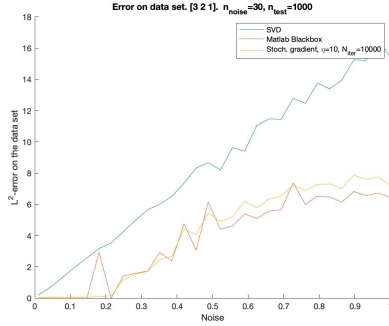


(a) Error on the dataset.

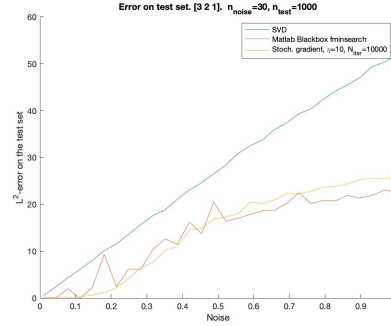


(b) Error on the test set.

Figure 8: Comparison of the SVD, and neural networks of shape $[3, 1]$ trained by fminsearch and stoch. gradient ($\eta = 10$, $N_{iter} = 1000$).



(a) Error on the dataset.

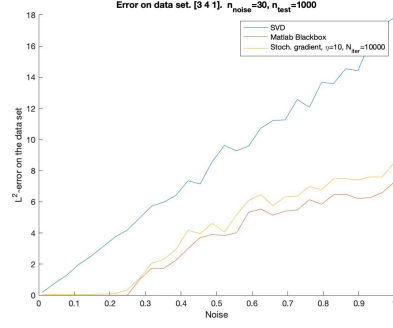


(b) Error on the test set.

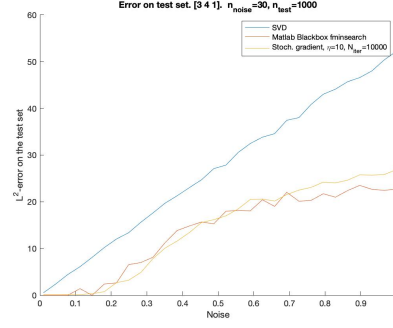
Figure 9: Comparison of the SVD, and neural networks of shape $[3, 2, 1]$ trained by fminsearch and stoch. gradient ($\eta = 10$, $N_{iter} = 1000$).

Figures 8, 9, 10, 11, 12 and 13 show the errors on the data of training and on the test set, for different shapes. As we can see, even though fminsearch has a lower error on the dataset, stochastic gradient has the same error on the test set. The neural networks seem performing better predictions compared to the SVD. Moreover, the neural networks seem to be less sensitive to high noise. The few irregularities are due to the randomness of the data and sometimes the non convergence of fminsearch.

Even though Section 4 showed a link between the optimal weights and the neural networks, this link seems not true when a non linear function is applied at the output layer and the neural networks can perform much better compared to the classical algorithms derived from linear algebra.

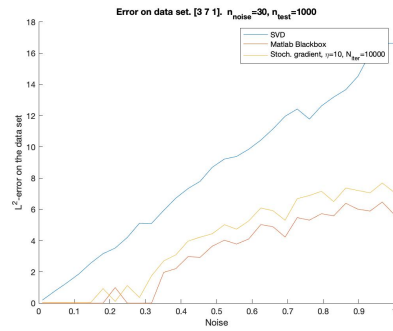


(a) Error on the dataset.

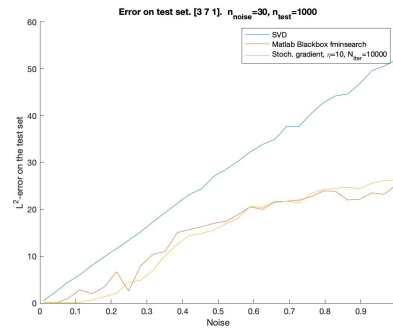


(b) Error on the test set.

Figure 10: Comparison of the SVD, and neural networks of shape $[3, 4, 1]$ trained by fminsearch and stoch. gradient ($\eta = 10$, $N_{iter} = 1000$).

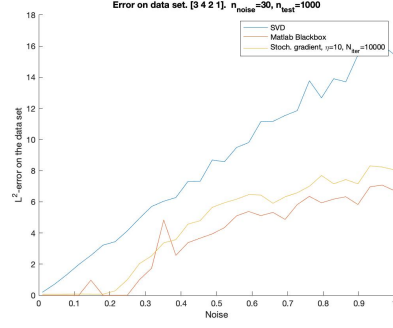


(a) Error on the dataset.

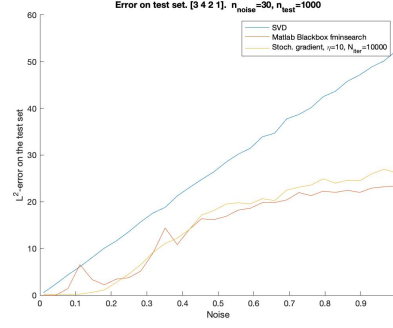


(b) Error on the test set.

Figure 11: Comparison of the SVD, and neural networks of shape $[3, 7, 1]$ trained by fminsearch and stoch. gradient ($\eta = 10$, $N_{iter} = 1000$).

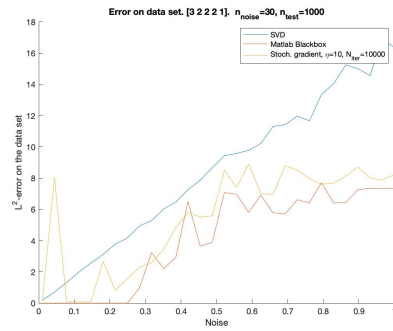


(a) Error on the dataset.

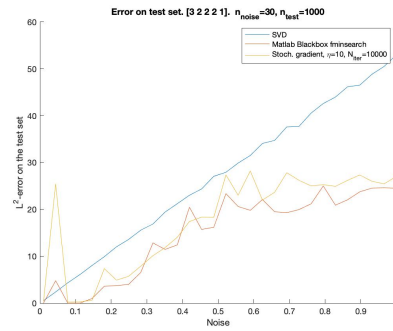


(b) Error on the test set.

Figure 12: Comparison of the SVD, and neural networks of shape $[3, 4, 2, 1]$ trained by fminsearch and stoch. gradient ($\eta = 10$, $N_{iter} = 1000$).



(a) Error on the dataset.



(b) Error on the test set.

Figure 13: Comparison of the SVD, and neural networks of shape $[3, 2, 2, 2, 1]$ trained by fminsearch and stoch. gradient ($\eta = 10$, $N_{iter} = 10000$).



Figure 14: Representation in MATLAB of the simplified model of Clash of Clans. Here, there are 8 troops and 5 towers. The troops move toward the towers at the same speed. Based on the rules described, the towers win.

5.2 Clash of Clans

5.2.1 The game

Clash of Clans is a popular game edited by Supercell, which is available on most recent smartphones. A basic principle in the game is to attack the villages of other players to steal their resources. The functioning of an attack is very simple. We have some troops available, and the towers of the enemy's village try to defend the village. A certain type of troop is efficient versus a certain type of tower, and a certain type of tower is efficient on a certain type of troop. The player has only to place his troops in the enemy's village, and then the rules do the job. Hence, the game is very easy to simulate. Of course, we don't have a super computer so we need to work on small data, and the purpose is mainly mathematical, so a simple model suffices.

We created a similar game in MATLAB, which is like clash of clans, in 1 dimension. We consider only two types of troops: the dots and the crosses. We also have two types of towers: the squares and the circles. The dots destroy the circles but die in the fight. The dots just die versus the squares. The crosses destroy the squares but die in the fight. The crosses just die versus the circles. The following table is a summary of the rules of the game. The goal is to destroy all the towers. Hence, if N_{towers} is the number of towers and N_{troops} the number of troops, we need to have $N_{troops} \geq N_{towers}$ to have a hope of winning.

	Tower : Circle	Tower : Square
Troop : Dot	Both destroyed	Tower's winning
Troop : Cross	Tower's winning	Both destroyed

To give an idea of what it looks like in MATLAB, Figure 14 represents a game generated randomly with 8 troops and 5 towers. The dot troop on the square tower will die and in the next step, the following dot will also fight the square tower and also die and so on. In this particular game, the towers win.

We want to create a neural network that learns to win. First, we have to translate the problem in a way that can be interpreted by a neural network, i.e.,

with a dataset of couples of input and output. In the following, we will focus on two approaches.

1. The first approach is a probabilistic approach. Given the places of troops and towers, what is the probability of winning ? The inputs are lists of troops and towers, containing their places and types. The outputs are win or loose.
2. The second approach is more what we expect from an AI. Given the places of towers, where do we place the troops to win the game ? The inputs are the places and types of each tower. The outputs are the places and types of each troop.

It is clear that both approaches can be generalized to more complex versions of the game, with more troops, towers and rules. Indeed, in the translation of the problem, we never used the rules of the game. It is easy to imagine doing it with places in two dimensions and more possible types. Of course, the larger the freedom is, the larger the data need to be (or not, see [6]). We can't learn from what we don't experience.

5.2.2 Probabilistic approach

The idea of this approach is, given the places and types of troops and towers, predict if we win or lose. We are in one dimension so we don't need to explicitly give the places of each troop and tower : they are included in their places in the vector. Remember, we have two types of troops : the dots (represented by 0) and the crosses (represented by 1). Idem for towers : we have the circles (represented by 0) and the squares (represented by 1). We also fix the number of troops and towers. The troops and towers are ordered in the vectors such that the first troop fights the first tower, etc. For a visual representation, if we represent the situation of Figure 14, we get the two vectors

$$\begin{aligned} \text{Troops :} \quad & [0, 0, 0, 1, 1, 0, 0, 0] = [\text{dot}, \text{dot}, \text{dot}, \text{cross}, \text{cross}, \text{dot}, \text{dot}, \text{dot}], \\ \text{Towers :} \quad & [1, 1, 1, 0, 1] = [\text{square}, \text{square}, \text{square}, \text{circle}, \text{square}]. \end{aligned}$$

Then, the input in the neural network is simply the concatenation of the two vectors, ie, $[\text{troops}, \text{towers}] = [0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1]$. As we said before, in this run the towers win, so the player (us) loses. Hence, the output is 0. To simulate the data, we play n_{data} times, with random troops and towers. However, we need to be careful because the number of possible games is finite. If we take n_{data} too large, we risk to play all possible configurations, and the test set would contain only already played configurations. Moreover, because of the rules of the game, the winning probability for a random configuration depends on the number of troops and towers. It is important because we need to have enough wins and losses in the data if we expect the neural network to learn something. We will try out many possibilities in terms of number of troops and towers. The possible shapes for the network are also huge, so we tried many

possibilities and we kept the most interesting results. Note also that the learning rate and the number of iterations for stochastic gradient need some test via a monitoring of the evolution of the cost to be well chosen. Finally, because we try to classify the data (loose and win, 0 and 1), the most appropriate function is the sigmoid as defined before, i.e., $\sigma(x) = 1/(1 + e^{-x})$ since its output is between 0 and 1.

In order to measure the quality of the predictions, we will focus on four values. The true positive rate (TPR) which measures the rate of well predicted wins. The true negative rate (TNR) which measures the rate of well predicted losses. The positive predictive value (PPV) which measures the precision of the prediction. And, of course, the error itself, which is exactly the number of games with a bad prediction. In all cases, we consider a test set generated randomly, and $n_{test} = 100000$.

1.

n_{troops}	n_{towers}	$n_{possibilities}$	n_{data}	n_{win}	n_{loose}	Network
5	3	$2^{5+3} = 256$	100	47	53	[8, 4, 3, 1]

Shape [8, 4, 3, 1], Figure 15	TPR	TNR	PPV	Error	Time
Initial random weights	1	0	0.4996	50042	/
fminsearch	0.4182	0.7673	0.6420	40714	6.8968
Stoch. gradient $\times 1$, $\eta = 0.1, N_{iter} = 500000$	0.6732	0.8067	0.7766	25997	6.0074
Stoch. gradient $\times 100$, mean (std)	0.7435 (0.0573)	0.8079 (0.0439)	0.7955 (0.0353)	22428 (3285)	6.4991 (0.3521)

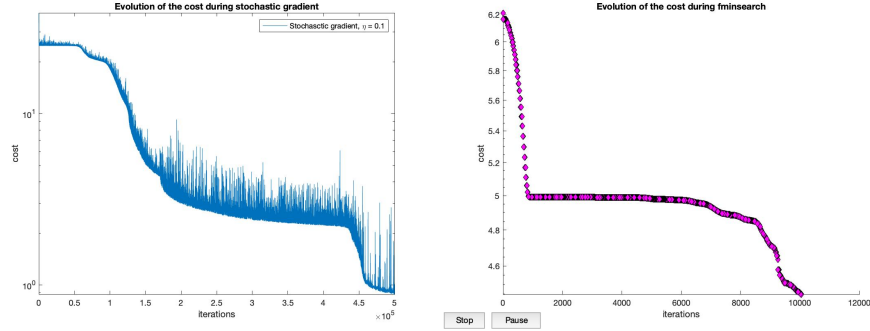
2.

n_{troops}	n_{towers}	$n_{possibilities}$	n_{data}	n_{win}	n_{loose}	Network
8	5	$2^{8+5} = 8192$	100	36	64	[13, 5, 1]

Shape [13, 5, 1], Figure 16	TPR	TNR	PPV	Error	Time
Initial random weights	1	0	0.3639	63613	/
fminsearch	0.3517	0.6890	0.3928	43376	3.0954
Stoch.gradient $\times 1$, $\eta = 0.5, N_{iter} = 100000$	0.4938	0.7676	0.5486	33203	1.0627
Stoch. gradient $\times 100$, mean (std)	0.4658 (0.0273)	0.7675 (0.0156)	0.5341 (0.0184)	34226 (1177)	0.8266 (0.0547)

3.

n_{troops}	n_{towers}	$n_{possibilities}$	n_{data}	n_{win}	n_{loose}	Network
8	5	$2^{8+5} = 8192$	500	170	330	[13, 5, 1]



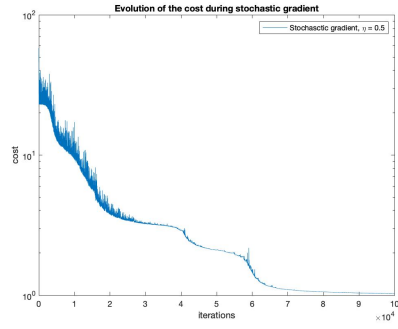
(a) Stoch. grad., $\eta = 0.1$, $N_{iter} = 500000$.

(b) fminsearch.

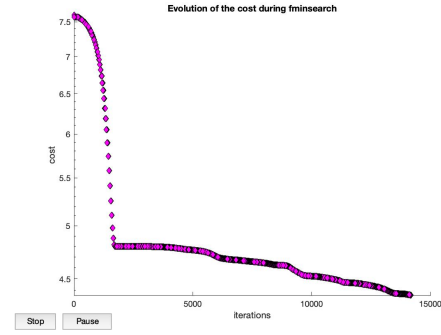
Figure 15: Evolution of the cost corresponding to results 1.

Shape [13, 5, 1], Figure 17	TPR	TNR	PPV	Error	Time
Initial random weights	1	0	0.3639	63613	/
fminsearch	0.2985	0.8355	0.5094	35988	44.4774
Stoch. gradient $\times 1$, $\eta = 0.5$, $N_{iter} = 100000$	0.5291	0.8304	0.6409	27923	1.1736
Stoch. gradient $\times 100$, mean (std)	0.5094 (0.0753)	0.8112 (0.0354)	0.6075 (0.0319)	29864 (2034)	0.7688 (0.0508)

- Note that problem 3 is the same as problem 2 with more data. The idea was to show the influence of the quantity of data on prediction. As we can expect, with more data the algorithms make less error. Observe also that the time of computation for stochastic gradient does not change compared to fminsearch. This comes from the fact that stochastic gradient never computes the cost, which is expensive because it depends on the whole data.
- As promised, we can observe that stochastic gradient performs better in much less time compare to fminsearch.

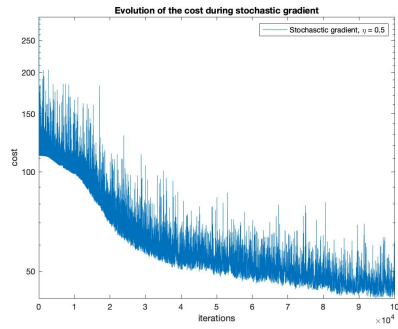


(a) Stoch. grad., $\eta = 0.5$, $N_{iter} = 100000$.

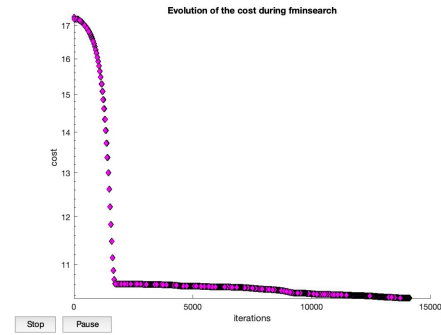


(b) fminsearch.

Figure 16: Evolution of the cost corresponding to results 2.



(a) Stoch. grad., $\eta = 0.5$, $N_{iter} = 100000$.



(b) fminsearch.

Figure 17: Evolution of the cost corresponding to results 3.

5.2.3 AI approach

The AI approach is more interesting because it is more realistic. In the game Clash of Clans, you just have the types and places of the enemy's towers and you have to choose the types and places of your troops.

In this approach, the inputs are the towers and the outputs are the troops. The sizes of inputs and outputs depend on the configuration of the game, i.e., the number of troops and towers you decide to use. In the real game, you cannot choose this. However, we can imagine to fix the sizes of input and output to the maximum size possible. Then, we can include the information of having less troops and towers by inserting 0. In fact, there are many possibilities and we have to decide by ourself what are the important informations that the neural network should see ?. It is clear that, in practice, the efficiency of an artificial neural network is very dependent on the way the information is modeled, i.e., the informations you give as input and the ones you expect as output.

In the previous approach, the information of winning was included in the output, and the artificial neural network was learning to predict if we win or lose. Now, we don't have any "winning parameter". But we want the neural network to learn how to win. Hence, when simulating the games, we keep in the dataset only the winning games. Note also that the number of possible configurations is much lower because we only consider the towers as input, so $n_{possibilities} = 2^{n_{towers}}$. Now, let's learn !

1. For the first run, we consider the problem with $n_{troops} = 10$ and $n_{towers} = 5$. Note that $n_{possibilities} = 2^5 = 32$ so we simulate only $n_{data} = 10$ generated randomly, keeping only the winning games. We train a neural network of shape $[5, 5, 10]$ (as before, we use the sigmoid function because the results are between 0 and 1). Then we simulate as test set $n_{test} = 100000$ games and we apply the trained neural network and count the winning games.

Shape $[5, 5, 10]$, Figure 18	Time	Number of wins
Initial random weights	/	3000
fminsearch	1.5400	40451
Stoch. gradient $\times 1$, $\eta = 0.5, N_{iter} = 10000$	0.3059	74890
Stoch. gradient $\times 100$, mean (std)	0.1312 (0.0188)	77296 (6130)

Stochastic gradient performs much better, in less time. Moreover, the standard deviation is not so high which is nice.

2. For the second run, we take the same problem but with a different shape of neural network, in order to inspect the influence on the time of computation and the predictions.

Shape [5, 20, 10], Figure 19	Time	Number of wins
Initial random weights	/	3000
fminsearch (no convergence)	9.4400	3000
Stoch. gradient $\times 1$, $\eta = 0.5$, $N_{iter} = 10000$	0.2919	77933
Stoch. gradient $\times 100$, mean (std)	0.1539 (0.0159)	77659 (3816)

As we can see, the influence on the time of computation for stochastic gradient is not significant with layers of this size. However, fminsearch does not converge anymore, which is probably due the properties of the Nelder-Mead simplex : it does not work well for high dimensional problems. Note also that the standard deviation is lower compared to the previous results. It means that a bigger network is more "stable" in terms of randomness.

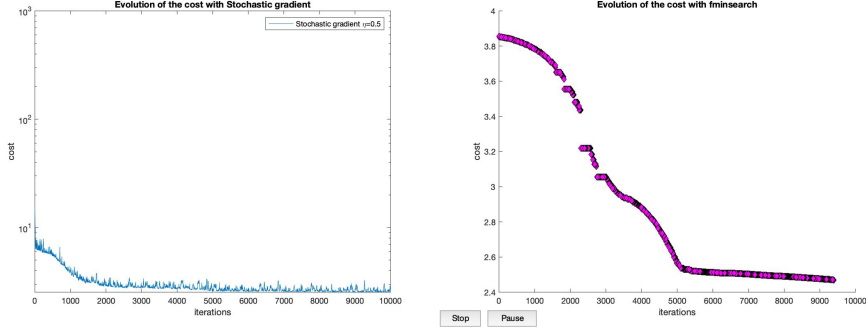
3. For the last run, we take $n_{troops} = 12$ and $n_{towers} = 12$. This means we have $n_{possibilities} = 2^{12} = 4096$ and we simulate $n_{data} = 100$ generated randomly, and keeping only the winning games. The neural network has shape [12, 12].

Shape [12, 12], Figure 20	Time	Number of wins
Initial random weights	/	22
fminsearch (no convergence)	3.53	22
Stoch. gradient $\times 1$, $\eta = 0.5$, $N_{iter} = 10000$	0.1353	100000
Stoch. gradient $\times 100$, mean (std)	0.0694 (0.0073)	100000 (0)

The results are stunning : stochastic gradient converges very rapidly and makes perfect prediction ! It is absolutely not the case for fminsearch, which does not even converge. This non-convergence is, again, probably due to the high dimensionality of the problem. But how is perfect prediction possible ? Let's inspect the prediction made on one random tower configuration :

Random towers :	[0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1]
Prediction by stochastic gradient :	[0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1]
Prediction by fminsearch :	[1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1]

To win the game, we need to place the troops exactly like the towers are placed. Hence, we want the input being equal to the output. Here, the prediction made by the neural network trained by stochastic gradient wins.



(a) Stoch. grad., $\eta = 0.5$, $N_{iter} = 10000$.

(b) fminsearch.

Figure 18: Evolution of the cost with the AI approach, $n_{data} = 100$, $n_{troops} = 12$, $n_{towers} = 12$.

It is not the case for fminsearch. In this case, the problem is easy. Since the number of troops and towers are equal, there is only one possibility to win : choose exactly the same troops. In other words, the relation between input and output is just the identity. And this is what is approximatively found by stochastic gradient. Indeed, the weighted matrix and bias are

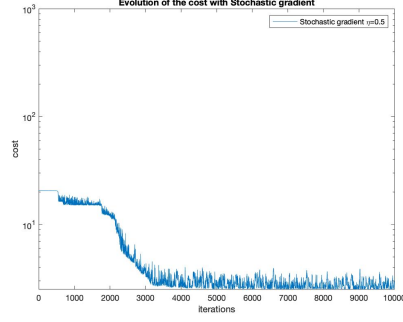
$$W_2 \approx 10 * I_{12} \in \mathbb{R}^{12 \times 12} \text{ and } b_2 \approx -4 \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^{12},$$

$$\sigma(10 - 4) = \sigma(6) \approx 1 \text{ and } \sigma(-4) \approx 0,$$

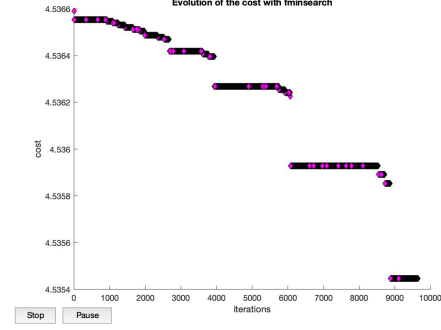
which finally produces an identity-like application ! Indeed,

$$NeuralNetwork(x) = F(x) = \sigma(W_2x + b_2) \approx x,$$

where the last approximation holds because x contains only 0 and 1.

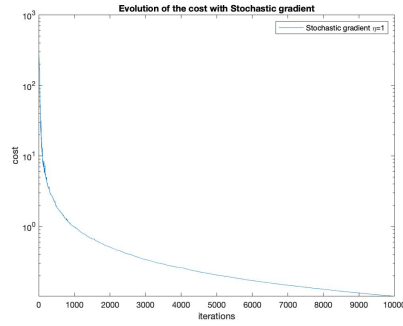


(a) Stoch. grad., $\eta = 0.5$, $N_{iter} = 10000$.

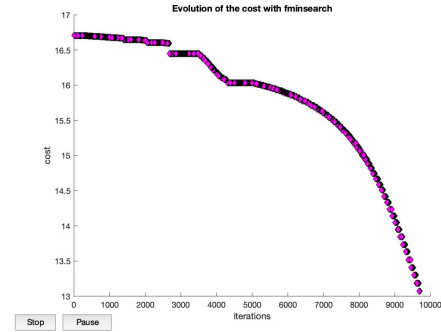


(b) fminsearch.

Figure 19: Evolution of the cost with the AI approach, $n_{data} = 100$, $n_{troops} = 12$, $n_{towers} = 12$.



(a) Stoch. grad., $\eta = 1$, $N_{iter} = 10000$.



(b) fminsearch.

Figure 20: Evolution of the cost with the AI approach, $n_{data} = 100$, $n_{troops} = 12$, $n_{towers} = 12$.

6 Conclusion

After introducing artificial neural networks, we introduced stochastic gradient to train efficiently neural networks with a given data set. In article [2] the authors proved that, under certain conditions, the optimal weights of a neural network are derived from the singular value decomposition of the dataset. We illustrated the theorem numerically. Then we compared, for a classification problem, neural networks and linear algebra (PCA with classifier). Finally, we applied neural networks with a machine learning problem with the clash of clans mini-game. It appears that neural networks perform better compared to algorithms from linear algebra. Stochastic gradient allows to find good solutions in a reasonable time. Moreover, this algorithm allows also to pre-train a network, which is very convenient in practice. However, the performance of the neural networks are not very well understood at this point. Indeed, the minimum found is not global. The global minimum performs bad due to overfitting, see section 5. Since stochastic gradient is a random algorithm, we never know which minimum we'll find. Hence, we need to perform stochastic gradient many times (which costs a lot) and also use techniques like cross-validation to confirm the quality of the minimum found. Another problem is the quantity of data. Indeed, we need a lot of data to train the network, but in certain cases, too much data is also bad, see article [6]. Neural networks are a theoretical problem but may be a practical solution.

References

- [1] N. Ahmed and K. R. Rao. *Orthogonal Transforms for Digital Signal Processing*. Springer-Verlag, Berlin Heidelberg, 1975.
- [2] H. Bourlard and Y. Kamp. Auto-association by multilayer perceptrons and singular value decomposition. *Biological Cybernetics*, 59(4-5):291–294, September 1988.
- [3] P. Delsarte and Y. Kamp. Low rank matrices with a given sign pattern. *SIAM Journal on Discrete Mathematics*, 2(1):51–63, February 1989.
- [4] Gene Howard Golub. Least squares, singular values and matrix approximations. *Aplikace matematiky*, 13(1):44–51, 1968. Publisher: Institute of Mathematics, Academy of Sciences of the Czech Republic.
- [5] Catherine F. Higham and Desmond J. Higham. Deep Learning: An Introduction for Applied Mathematicians. *SIAM Review*, 61(3):860–891, January 2019.
- [6] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep Double Descent: Where Bigger Models and More Data Hurt. *arXiv:1912.02292 [cs, stat]*, December 2019. arXiv: 1912.02292.

- [7] G. W. Stewart. *Introduction to Matrix Computations*. Academic Press Inc, New York, June 1973.
- [8] Gilbert Strang. *Linear Algebra and Learning from Data*. Wellesley-Cambridge Press, Wellesley, MA, January 2019.